

Optimization-based Invariant Generation

Hao Wu

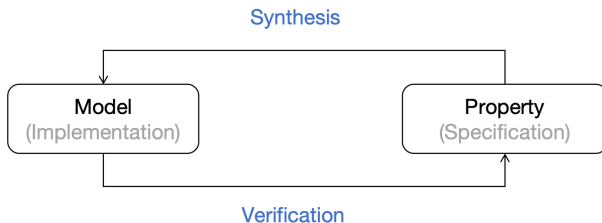
Institute of Software, CAS & UCAS

AMSS, Feb. 26, 2025

Contents

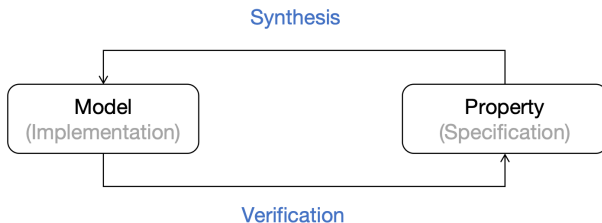
- 1 Introduction to Invariant Generation.
- 2 Optimization-based Invariant Generation
- 3 Complete SOS Constraints over Unbounded Domains
- 4 Strong Invariant Generation via Robust Optimization Techniques
- 5 Challenges

What is Verification?



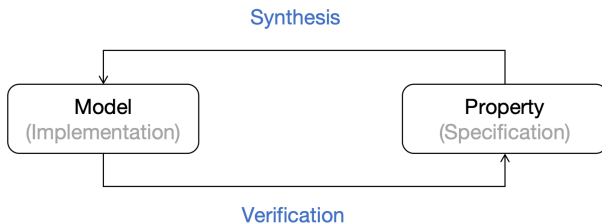
- **Verification**: to prove a **given model** satisfy a property.
- **Synthesis**: to generate a model subject to a **given property**.
- The research for verification is much more mature than synthesis.

What is Verification?



- **Verification**: to prove a **given model** satisfy a property.
- **Synthesis**: to generate a model subject to a **given property**.
- The research for verification is much more mature than synthesis.

What is Verification?



- **Verification**: to prove a **given model** satisfy a property.
- **Synthesis**: to generate a model subject to a **given property**.
- The research for verification is much more mature than synthesis.

Program Verification

- Consider a program C :

```
 $x \leftarrow 2$   
while True do  
   $x \leftarrow 2x - 1$   
end
```

- Question:** How to reason about a program? For example, to prove that $x > 0$ always holds.
- Hoare logic** by Hoare 1969.

Program Verification

- Consider a program C :

```
 $x \leftarrow 2$   
while True do  
   $x \leftarrow 2x - 1$   
end
```

- Question:** How to reason about a program? For example, to prove that $x > 0$ always holds.
- Hoare logic** by Hoare 1969.

Program Verification: Hoare Logic

- **Hoare Logic**: to insert **logical assertions** into programs:

```
{ }(Pre-condition)
 $x \leftarrow 2$ 
 $\{x = 2\}$ 
while True do
   $x \leftarrow 2x - 1$ 
end
 $\{x > 0\}$ (Post-condition)
```

- **Sub-Question**: How to reason about loops?
- The key of Hoare logic: **Loop (Inductive) Invariants**.

Program Verification: Hoare Logic

- **Hoare Logic**: to insert **logical assertions** into programs:

```
{ }(Pre-condition)
 $x \leftarrow 2$ 
 $\{x = 2\}$ 
while True do
   $x \leftarrow 2x - 1$ 
end
 $\{x > 0\}$ (Post-condition)
```

- **Sub-Question**: How to reason about loops?
- The key of Hoare logic: **Loop (Inductive) Invariants**.

Program Verification: Inductive Invariants

```
...  
{ $x = 2$ }  
while True do {?}  
   $x \leftarrow 2x - 1$   
end  
{ $x > 0$ }
```

- **Loop Invariant:** An assertion at the loop entrance that holds whenever the program reaches that point. (say, $x > 0$)
- **Loop Inductive Invariant:** A loop invariant that is **inductive**: If it holds in one iteration, then it still holds in the next iteration. (say, $x \geq 1$)

Program Verification: Verification Conditions

```

{Pre(x)}
while B(x) do {Inv(x)}
    x ← f(x)
end
{Post(x)}

```

- $Pre(x) \implies Inv(x)$
- $Inv(x) \wedge B(x) \implies Inv(f(x))$
- $Inv(x) \wedge \neg B(x) \implies Post(x)$
- To automatize program verification, one needs to find **suitable invariants** for each loop.

Program Verification: Verification Conditions

```

{Pre(x)}
while B(x) do {Inv(x)}
    x ← f(x)
end
{Post(x)}

```

- $Pre(x) \implies Inv(x)$
- $Inv(x) \wedge B(x) \implies Inv(f(x))$
- $Inv(x) \wedge \neg B(x) \implies Post(x)$
- To automatize program verification, one needs to find **suitable invariants** for each loop.

Extending to Continuous-Time Dynamical Systems

System dynamics given by Ordinary Differential Equations (ODEs):

$$\dot{x} = f(x)$$

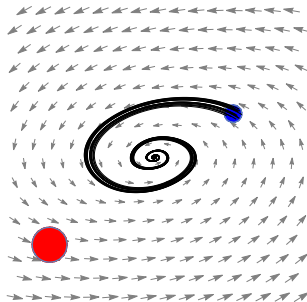
- $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ is a **locally Lipschitz continuous** vector field.
- Unique **trajectory** $\xi_{x_0} : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}^n$ for any initial state x_0 .

Safety Verification Problem of ODE Systems

Safety Verification Problem: Given domain $\mathcal{X}_d \subseteq \mathbb{R}^n$, initial set $\mathcal{X}_i \subset \mathcal{X}_d$, and unsafe set $\mathcal{X}_u \subset \mathcal{X}_d$ such that $\mathcal{X}_u \cap \mathcal{X}_i = \emptyset$. Prove that

$\mathcal{X}_u$ is **not reachable** from any state in $\mathcal{X}_i$.

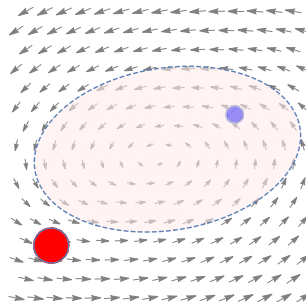
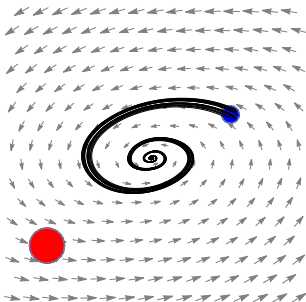
- $\mathcal{X}_i$: Pre-condition.
- $\mathcal{X}_u$: negation of Post-condition.
- $\mathcal{X}_d$: variable range



Differential Invariants

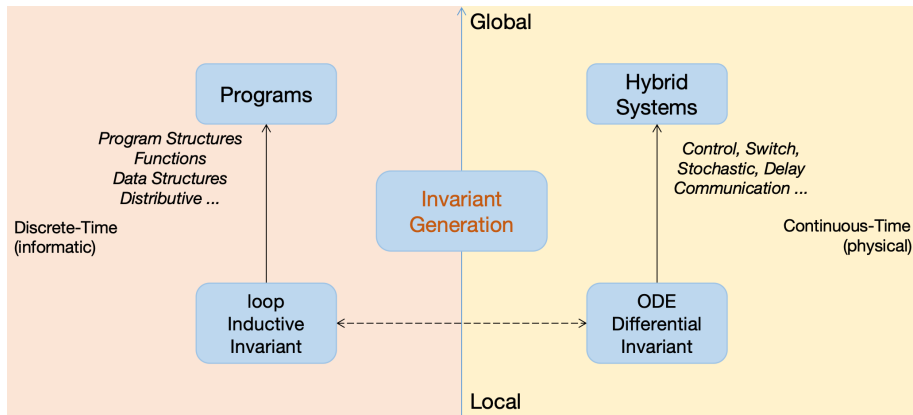
- Analogous to **inductive invariants** for programs.
- A **differential/positive invariant** is a subset $\Omega \subseteq \mathcal{X}_d$ such that any trajectory starting from Ω stays within Ω forever, i.e.,

$$\forall x_0 \in \Omega, \forall t \in \mathbb{R}_{\geq 0}. \xi_{x_0}(t) \in \Omega.$$



Invariant Generation

- The **most challenging problem** for **Hoare-logic**-style verification.



- 1 Introduction to Invariant Generation.
- 2 Optimization-based Invariant Generation**
- 3 Complete SOS Constraints over Unbounded Domains
- 4 Strong Invariant Generation via Robust Optimization Techniques
- 5 Challenges

Optimization-based Invariant Generation

- Math optimization provides a **unified framework** for synthesizing inductive invariants and differential invariants.
 - ① Set a parameterized template for the target invariant.
 - ② Encode the invariant conditions into constraints for parameters.
 - ③ Solve the constraints to obtain assignments to parameters.
- Assumptions:
 - ① All functions and predicates are **polynomial or semialgebraic**.
 - ② The invariant template is a **polynomial with unknown coefficients** and the target invariant is expressed as $I(\mathbf{x}; \mathbf{a}) \leq 0$. For example

$$I(\mathbf{x}; \mathbf{a}) = a_1x_1^2 + a_2x_1x_2 + a_3x_2^2 + a_4x_1 + a_5x_2 + a_6 \in \mathbb{R}[x_1, x_2],$$

Invariant Conditions

- We consider pre-condition, post-condition, domain, initial set, and unsafe set are all **basic semialgebraic sets**.
- Conditions for **inductive invariants**:
 - 1 $\forall \mathbf{x}. \mathbf{x} \in \text{Pre} \implies I(\mathbf{x}; \mathbf{a}) \leq 0,$
 - 2 $\forall \mathbf{x}. I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) \leq 0 \implies I(f(\mathbf{x}); \mathbf{a}) \leq 0,$
 - 3 $\forall \mathbf{x}. I(\mathbf{x}; \mathbf{a}) \leq 0 \wedge g(\mathbf{x}) > 0 \implies \mathbf{x} \in \text{Post}.$
- (Sufficient) Conditions for **differential invariants** (or **barrier certificates**):
 - 1 $\forall \mathbf{x}. \mathbf{x} \in \mathcal{X}_i \implies B(\mathbf{x}; \mathbf{a}) \leq 0,$
 - 2 $\forall \mathbf{x}. \mathbf{x} \in \mathcal{X}_d \wedge B(\mathbf{x}; \mathbf{a}) = 0 \implies \mathcal{L}_f B(\mathbf{x}; \mathbf{a}) \leq 0,$
 - 3 $\forall \mathbf{x}. \mathbf{x} \in \mathcal{X}_u \implies B(\mathbf{x}; \mathbf{a}) > 0.$

where $\mathcal{L}_f B(\mathbf{x}; \mathbf{a}) \triangleq \langle \nabla B(\mathbf{x}; \mathbf{a}), f(\mathbf{x}) \rangle$ is the Lie derivative.

Invariant Conditions

- We consider pre-condition, post-condition, domain, initial set, and unsafe set are all **basic semialgebraic sets**.
- Conditions for **inductive invariants**:
 - ① $\forall x. x \in \text{Pre} \implies I(x; a) \leq 0,$
 - ② $\forall x. I(x; a) \leq 0 \wedge g(x) \leq 0 \implies I(f(x); a) \leq 0,$
 - ③ $\forall x. I(x; a) \leq 0 \wedge g(x) > 0 \implies x \in \text{Post}.$
- (Sufficient) Conditions for **differential invariants** (or **barrier certificates**):
 - ① $\forall x. x \in \mathcal{X}_i \implies B(x; a) \leq 0,$
 - ② $\forall x. x \in \mathcal{X}_d \wedge B(x; a) = 0 \implies \mathcal{L}_f B(x; a) \leq 0,$
 - ③ $\forall x. x \in \mathcal{X}_u \implies B(x; a) > 0.$

where $\mathcal{L}_f B(x; a) \triangleq \langle \nabla B(x; a), f(x) \rangle$ is the Lie derivative.

SOS polynomials

- A polynomial $\sigma(\mathbf{x})$ is **Sum-of-Squares** (SOS) if it can be written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^n p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\mathbf{x})$ is SOS iff the **Gram matrix** of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_1x_2^3 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}}' \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^2 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

SOS polynomials

- A polynomial $\sigma(\mathbf{x})$ is **Sum-of-Squares** (SOS) if it can be written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^n p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\mathbf{x})$ is SOS iff the **Gram matrix** of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_1^2x_2^2 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}}' \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^2 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

SOS polynomials

- A polynomial $\sigma(\mathbf{x})$ is **Sum-of-Squares** (SOS) if it can be written as

$$\sigma(\mathbf{x}) = \sum_{i=1}^n p_i^2(\mathbf{x}) \in \mathbb{R}[\mathbf{x}].$$

- $\Sigma[\mathbf{x}]$ denotes the set of SOS polynomials.
- A polynomial $f(\mathbf{x})$ is SOS iff the **Gram matrix** of f is PSD.

$$\begin{aligned} f(x_1, x_2) &= 2x_1^4 + 2x_1^3x_2 - x_1^2x_2^2 + 5x_2^4 \\ &= \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}}' \underbrace{\begin{pmatrix} 2 & -3 & 1 \\ -3 & 5 & 0 \\ 1 & 0 & 5 \end{pmatrix}}_{\text{Gram matrix}} \underbrace{\begin{pmatrix} x_1^2 \\ x_2^2 \\ x_1x_2 \end{pmatrix}}_{\text{basis}} \\ &= \frac{1}{2}(2x_1^2 - 3x_2^2 + x_1x_2)^2 + \frac{1}{2}(x_2^2 + 3x_1x_2)^2 \end{aligned}$$

Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0, i = 1, \dots, m\}$.
- **Quadratic Module**

$$\mathbf{QM}(S) \triangleq \left\{ \sigma_0 + \sum_{i=1}^m \sigma_i p_i \mid \sigma_i \in \Sigma[x] \text{ for } 1 \leq i \leq m \right\}$$

- A quadratic module $\mathbf{QM}$ is **Archimedean** if $N - \|x\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
 - $N - \|x\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is **strictly positive** on S , then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^m p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0, i = 1, \dots, m\}$.
- **Quadratic Module**

$$\mathbf{QM}(S) \triangleq \left\{ \sigma_0 + \sum_{i=1}^m \sigma_i p_i \mid \sigma_i \in \Sigma[x] \text{ for } 1 \leq i \leq m \right\}$$

- A quadratic module **QM** is **Archimedean** if $N - \|x\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
 - $N - \|x\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is *strictly positive* on S , then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^m p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Putinar's Positivstellensatz

- Let $S = \{x \mid p_i(x) \geq 0, i = 1, \dots, m\}$.
- Quadratic Module**

$$\mathbf{QM}(S) \triangleq \left\{ \sigma_0 + \sum_{i=1}^m \sigma_i p_i \mid \sigma_i \in \Sigma[x] \text{ for } 1 \leq i \leq m \right\}$$

- A quadratic module $\mathbf{QM}$ is **Archimedean** if $N - \|x\|^2 \in \mathbf{QM}$ for some constant $N \in \mathbb{N}$.
 - $N - \|x\|^2 \geq 0$ over S implies that S is bounded.

Theorem (Putinar's Positivstellensatz)

Assume the Archimedean condition holds. If $f \in \mathbb{R}[x]$ is *strictly positive* on S , then $f \in \mathbf{QM}(S)$, i.e.,

$$f(x) = \sigma_0(x) + \sum_{i=1}^m p_i(x) \cdot \sigma_i(x), \quad \sigma_i(x) \in \Sigma[x].$$

Sum-of-Squares Constraints

- Take differential invariants for example:

- 1 $\forall x. x \in \mathcal{X}_i \implies B(x; a) \leq 0,$

- 2 $\forall x. x \in \mathcal{X}_d \wedge B(x; a) = 0 \implies \mathcal{L}_f B(x; a) \leq 0,$

- 3 $\forall x. x \in \mathcal{X}_u \implies B(x; a) > 0.$

- By applying Putinar's Positivstellensatz

find a, b

$$-B(x; a) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(x; b)B(x; a) - \mathcal{L}_f B(x; a) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(x; a) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- When $\lambda(x)$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ **non-convex** and **NP-hard**.

Sum-of-Squares Constraints

- Take differential invariants for example:
 - $\forall x. x \in \mathcal{X}_i \implies B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_d \wedge B(x; a) = 0 \implies \mathcal{L}_f B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_u \implies B(x; a) > 0.$
- By applying Putinar's Positivstellensatz

find a, b

$$-B(x; a) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(x; b)B(x; a) - \mathcal{L}_f B(x; a) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(x; a) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- When $\lambda(x)$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ **non-convex** and **NP-hard**.

Sum-of-Squares Constraints

- Take differential invariants for example:
 - $\forall x. x \in \mathcal{X}_i \implies B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_d \wedge B(x; a) = 0 \implies \mathcal{L}_f B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_u \implies B(x; a) > 0.$
- By applying Putinar's Positivstellensatz

find a, b

$$-B(x; a) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(x; b)B(x; a) - \mathcal{L}_f B(x; a) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(x; a) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- When $\lambda(x)$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ non-convex and NP-hard.

Sum-of-Squares Constraints

- Take differential invariants for example:
 - $\forall x. x \in \mathcal{X}_i \implies B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_d \wedge B(x; a) = 0 \implies \mathcal{L}_f B(x; a) \leq 0,$
 - $\forall x. x \in \mathcal{X}_u \implies B(x; a) > 0.$
- By applying Putinar's Positivstellensatz

find a, b

$$-B(x; a) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(x; b)B(x; a) - \mathcal{L}_f B(x; a) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(x; a) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- When $\lambda(x)$ is known, **linear matrix inequalities (LMIs)** $\implies$ solved by **SDP solvers** efficiently.
- In general, **bilinear matrix inequalities (BMIs)** $\implies$ **non-convex** and **NP-hard**.

Our Works

- **Problem:** Using Putinar's Positivstellensatz requires that the domain is bounded. How to deal with **unbounded problems**?
- Our FM work [Wu et al., 2024], based on [Huang et al., 2023].
- **Problem:** How to solve the **strong invariant generation problem**, i.e., characterizing the set of all valid parameter assignments?
- Our TOPLAS work [Wu et al., 2025], based on [Lasserre, 2015].

Our Works

- **Problem:** Using Putinar's Positivstellensatz requires that the domain is bounded. How to deal with **unbounded problems**?
- Our FM work [Wu et al., 2024], based on [Huang et al., 2023].
- **Problem:** How to solve the **strong invariant generation problem**, i.e., characterizing the set of all valid parameter assignments?
- Our TOPLAS work [Wu et al., 2025], based on [Lasserre, 2015].

- 1 Introduction to Invariant Generation.
- 2 Optimization-based Invariant Generation
- 3 Complete SOS Constraints over Unbounded Domains**
- 4 Strong Invariant Generation via Robust Optimization Techniques
- 5 Challenges

Algebraic Tool: Homogenization

- Idea: project the **unbounded** domain into a **bounded** region.
- Introduce a fresh variable x_0 . Denote $\tilde{x} = (x_0, x) \in \mathbb{R}^{n+1}$.
- Let $p(x)$ be a polynomial of degree d , its **homogenization** is

$$\tilde{p}(\tilde{x}) \triangleq x_0^d \cdot p\left(\frac{x_1}{x_0}, \dots, \frac{x_n}{x_0}\right).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{x \in \mathbb{R}^n \mid p_1(x) \geq 0, \dots, p_m(x) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{x} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{x}) \geq 0, \dots, \tilde{p}_m(\tilde{x}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{x}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Idea: project the **unbounded** domain into a **bounded** region.
- Introduce a fresh variable x_0 . Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d , its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \triangleq x_0^d \cdot p\left(\frac{x_1}{x_0}, \dots, \frac{x_n}{x_0}\right).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Idea: project the **unbounded** domain into a **bounded** region.
- Introduce a fresh variable x_0 . Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d , its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \triangleq x_0^d \cdot p\left(\frac{x_1}{x_0}, \dots, \frac{x_n}{x_0}\right).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

- Idea: project the **unbounded** domain into a **bounded** region.
- Introduce a fresh variable x_0 . Denote $\tilde{\mathbf{x}} = (x_0, \mathbf{x}) \in \mathbb{R}^{n+1}$.
- Let $p(\mathbf{x})$ be a polynomial of degree d , its **homogenization** is

$$\tilde{p}(\tilde{\mathbf{x}}) \triangleq x_0^d \cdot p\left(\frac{x_1}{x_0}, \dots, \frac{x_n}{x_0}\right).$$

$$p(x_1, x_2) = x_1^2 + x_2 + 1 \implies \tilde{p}(x_0, x_1, x_2) = x_1^2 + x_2 x_0 + x_0^2.$$

- Let $S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\}$, define

$$\tilde{S} \triangleq \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \underbrace{x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1}_{\text{half unit sphere in } \mathbb{R}^{n+1}}\}.$$

Algebraic Tool: Homogenization

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\},$$

$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$

$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \mathbf{x}_0 > 0, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- An one-to-one projection between S and $\tilde{S}_+$:

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \dots, \frac{x_n}{\sqrt{1 + \|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to points at infinity in $\mathbb{R}^n$.
- For these points, a technical assumption is needed.

Algebraic Tool: Homogenization

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\},$$

$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$

$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \textcolor{red}{x}_0 > \textcolor{red}{0}, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- An **one-to-one projection** between S and $\tilde{S}_+$:

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \dots, \frac{x_n}{\sqrt{1 + \|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- For these points, a **technical assumption** is needed.

Algebraic Tool: Homogenization

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\},$$

$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$

$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \textcolor{red}{x}_0 > \textcolor{red}{0}, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- An **one-to-one projection** between S and $\tilde{S}_+$:

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \dots, \frac{x_n}{\sqrt{1 + \|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- For these points, a **technical assumption** is needed.

Algebraic Tool: Homogenization

$$S = \{\mathbf{x} \in \mathbb{R}^n \mid p_1(\mathbf{x}) \geq 0, \dots, p_m(\mathbf{x}) \geq 0\},$$

$$\tilde{S} = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, x_0 \geq 0, \|\tilde{\mathbf{x}}\|^2 = 1\},$$

$$\tilde{S}_+ = \{\tilde{\mathbf{x}} \in \mathbb{R}^{n+1} \mid \tilde{p}_1(\tilde{\mathbf{x}}) \geq 0, \dots, \tilde{p}_m(\tilde{\mathbf{x}}) \geq 0, \textcolor{red}{x}_0 > \textcolor{red}{0}, \|\tilde{\mathbf{x}}\|^2 = 1\}.$$

- An **one-to-one projection** between S and $\tilde{S}_+$:

$$\mathbf{x} \in S \iff \left(\frac{1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1 + \|\mathbf{x}\|^2}}, \dots, \frac{x_n}{\sqrt{1 + \|\mathbf{x}\|^2}} \right) \in \tilde{S}_+.$$

- Points with $x_0 = 0$ in $\tilde{S}$ correspond to **points at infinity** in $\mathbb{R}^n$.
- For these points, a **technical assumption** is needed.

Algebraic Tool: Homogenization

Definition (closed at infinity (Nie, 2012))

A basic semialgebraic set S is closed at ∞ if

$$cl(\tilde{S}_+) = \tilde{S}.$$

- Being closure at infinity is a generic property for semialgebraic sets.
- Depends on description polynomials.

$$S_1 = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 - x_2^2 \geq 0\} \text{ not closed at } \infty$$

$$S_2 = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 - x_2^2 \geq 0, \underbrace{x_1 \geq 0}_{\text{redundant}}\} \text{ closed at } \infty$$

Algebraic Tool: Homogenization

Theorem (Equivalence (Huang et al., 2023))

When S is closed at ∞ ,

$$f(\mathbf{x}) \geq 0 \text{ over } S \iff \tilde{f}(\tilde{\mathbf{x}}) \geq 0 \text{ over } \tilde{S}$$

- The quadratic module $\mathbf{QM}(\tilde{S})$ is Archimedean.

Constraints for Polynomial BC

- Old SOS constraints for exponential-type conditions:

$$\begin{aligned} -B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_i), \\ B(\mathbf{x}) - \epsilon_e &\in \mathbf{QM}(\mathcal{X}_u) \\ \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_d) \end{aligned}$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$\begin{aligned} -\tilde{B}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_i) \\ \tilde{B}(x_0, \mathbf{x}) - \epsilon_e x_0^d &\in \mathbf{QM}(\tilde{\mathcal{X}}_u) \\ \tilde{H}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_d) \end{aligned}$$

where $H(\mathbf{x}) = \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all **closed at infinity**. Under some mild conditions, both **sound** and **complete**.

Constraints for Polynomial BC

- Old SOS constraints for exponential-type conditions:

$$\begin{aligned} -B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_i), \\ B(\mathbf{x}) - \epsilon_e &\in \mathbf{QM}(\mathcal{X}_u) \\ \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_d) \end{aligned}$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$\begin{aligned} -\tilde{B}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_i) \\ \tilde{B}(x_0, \mathbf{x}) - \epsilon_e x_0^d &\in \mathbf{QM}(\tilde{\mathcal{X}}_u) \\ \tilde{H}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_d) \end{aligned}$$

where $H(\mathbf{x}) = \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all **closed at infinity**. Under some mild conditions, both **sound** and **complete**.

Constraints for Polynomial BC

- Old SOS constraints for exponential-type conditions:

$$\begin{aligned} -B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_i), \\ B(\mathbf{x}) - \epsilon_e &\in \mathbf{QM}(\mathcal{X}_u) \\ \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x}) &\in \mathbf{QM}(\mathcal{X}_d) \end{aligned}$$

- First **homogenization**, then **Putinar's Positivstellensatz**.

$$\begin{aligned} -\tilde{B}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_i) \\ \tilde{B}(x_0, \mathbf{x}) - \epsilon_e x_0^d &\in \mathbf{QM}(\tilde{\mathcal{X}}_u) \\ \tilde{H}(x_0, \mathbf{x}) &\in \mathbf{QM}(\tilde{\mathcal{X}}_d) \end{aligned}$$

where $H(\mathbf{x}) = \lambda B(\mathbf{x}) - \mathfrak{L}_f B(\mathbf{x})$.

- Assume that $\mathcal{X}_i$, $\mathcal{X}_u$, and $\mathcal{X}_d$ are all **closed at infinity**. Under some mild conditions, both **sound** and **complete**.

From CS perspective...

- What we have done:
 - ① Project a system in $\mathbb{R}^n$ into a **new** bounded system over in $\mathbb{R}^{n+1}$.
 - ② Construct constraints over the **new** system.
 - ③ Solve constraints to obtain a polynomial BC for the **original** system.
- **Question:** Can we just work on the new system?
- Let $(y_0, y_1, \dots, y_n) = (\frac{1}{\sqrt{1+\|x\|^2}}, \frac{x_1}{\sqrt{1+\|x\|^2}}, \dots, \frac{x_n}{\sqrt{1+\|x\|^2}})$ be the state vector of the new system.

Theorem

$B(y_0, \mathbf{y}) \in \mathcal{C}^1(\mathbb{R}^{n+1})$ is a barrier certificate of the homogenized system if and only if $B(\frac{1}{\sqrt{\|x\|^2+1}}, \frac{\mathbf{x}}{\sqrt{\|x\|^2+1}})$ is a barrier certificate of the original system.

From CS perspective...

- What we have done:
 - ① Project a system in $\mathbb{R}^n$ into a **new** bounded system over in $\mathbb{R}^{n+1}$.
 - ② Construct constraints over the **new** system.
 - ③ Solve constraints to obtain a polynomial BC for the **original** system.
- **Question:** Can we just work on the new system?
- Let $(y_0, y_1, \dots, y_n) = (\frac{1}{\sqrt{1+\|x\|^2}}, \frac{x_1}{\sqrt{1+\|x\|^2}}, \dots, \frac{x_n}{\sqrt{1+\|x\|^2}})$ be the state vector of the new system.

Theorem

$B(y_0, \mathbf{y}) \in \mathcal{C}^1(\mathbb{R}^{n+1})$ is a barrier certificate of the homogenized system if and only if $B(\frac{1}{\sqrt{\|x\|^2+1}}, \frac{\mathbf{x}}{\sqrt{\|x\|^2+1}})$ is a barrier certificate of the original system.

From CS perspective...

- What we have done:
 - ① Project a system in $\mathbb{R}^n$ into a **new** bounded system over in $\mathbb{R}^{n+1}$.
 - ② Construct constraints over the **new** system.
 - ③ Solve constraints to obtain a polynomial BC for the **original** system.
- **Question:** Can we just work on the new system?
- Let $(y_0, y_1, \dots, y_n) = (\frac{1}{\sqrt{1+\|\mathbf{x}\|^2}}, \frac{x_1}{\sqrt{1+\|\mathbf{x}\|^2}}, \dots, \frac{x_n}{\sqrt{1+\|\mathbf{x}\|^2}})$ be the state vector of the new system.

Theorem

$B(y_0, \mathbf{y}) \in \mathcal{C}^1(\mathbb{R}^{n+1})$ is a barrier certificate of the homogenized system if and only if $B(\frac{1}{\sqrt{\|\mathbf{x}\|^2+1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2+1}})$ is a barrier certificate of the original system.

Semialgebraic Barrier Certificates

- To avoid rational functions, we simplified by

$$\begin{aligned}
 & (\sqrt{\|\mathbf{x}\|^2 + 1})^d \cdot B\left(\frac{1}{\sqrt{\|\mathbf{x}\|^2 + 1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2 + 1}}\right) \\
 &= \underbrace{B_1(\mathbf{x}) + \sqrt{\|\mathbf{x}\|^2 + 1} \cdot B_2(\mathbf{x})}_{\text{semialgebraic BC}}
 \end{aligned}$$

where $B_1, B_2 \in \mathbb{R}[\mathbf{x}]$.

- Semialgebraic BC **subsumes** polynomial BC as a special case, where $B_2(\mathbf{x})$ is constant zero.
- Semialgebraic BC can also be synthesized using SDP.

Semialgebraic Barrier Certificates

- To avoid rational functions, we simplified by

$$\begin{aligned}
 & (\sqrt{\|\mathbf{x}\|^2 + 1})^d \cdot B\left(\frac{1}{\sqrt{\|\mathbf{x}\|^2 + 1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2 + 1}}\right) \\
 &= \underbrace{B_1(\mathbf{x}) + \sqrt{\|\mathbf{x}\|^2 + 1} \cdot B_2(\mathbf{x})}_{\text{semialgebraic BC}}
 \end{aligned}$$

where $B_1, B_2 \in \mathbb{R}[\mathbf{x}]$.

- Semialgebraic BC **subsumes** polynomial BC as a special case, where $B_2(\mathbf{x})$ is constant zero.
- Semialgebraic BC can also be synthesized using SDP.

Semialgebraic Barrier Certificates

- To avoid rational functions, we simplified by

$$\begin{aligned}
 & (\sqrt{\|\mathbf{x}\|^2 + 1})^d \cdot B\left(\frac{1}{\sqrt{\|\mathbf{x}\|^2 + 1}}, \frac{\mathbf{x}}{\sqrt{\|\mathbf{x}\|^2 + 1}}\right) \\
 &= \underbrace{B_1(\mathbf{x}) + \sqrt{\|\mathbf{x}\|^2 + 1} \cdot B_2(\mathbf{x})}_{\text{semialgebraic BC}}
 \end{aligned}$$

where $B_1, B_2 \in \mathbb{R}[\mathbf{x}]$.

- Semialgebraic BC **subsumes** polynomial BC as a special case, where $B_2(\mathbf{x})$ is constant zero.
- Semialgebraic BC can also be synthesized using SDP.

Experiments

system	dim	unbounded	Incomplete		Polynomial BC			Semialgebraic BC			
			deg	succ time(s)	deg	succ	time(s)	deg	succ	time(s)	
vector[37]	2	$\mathcal{X}$	4	✓	0.58	3	✓	0.03	2	✓	0.83
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	> 6	✗	0.39	4	✓	0.16	2	✓	0.72
barrier[29]	2	$\mathcal{X}$	> 6	✗	2.40	> 6	✗	2.73	> 4	✗	22.98
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	> 6	✗	0.78	3	✓	0.04	2	✓	3.12
lie-der[24]	2	$\mathcal{X}$	3	✓	0.19	3	✓	0.14	1	✓	0.75
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	3	✓	0.14	3	✓	0.19	3	✓	5.04
arch1[36]	2	$\mathcal{X}$	4	✓	0.40	4	✓	0.54	> 4	✗	117.61
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	1	✓	0.09	1	✓	0.04	2	✓	20.92
arch2[36]	2	$\mathcal{X}$	3	✓	0.20	3	✓	0.21	3	✓	5.53
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	3	✓	0.18	3	✓	0.18	2	✓	1.59
arch3[36]	2	$\mathcal{X}$	2	✓	0.12	2	✓	0.13	2	✓	3.45
		$\mathcal{I}, \mathcal{U}, \mathcal{X}$	> 6	✗	0.84	> 6	✗	1.30	1	✓	0.34
arch4[36]	2	$\mathcal{X}$	> 6	✗	0.11	5	✓	0.74	3	✓	5.69
		$\mathcal{U}, \mathcal{X}$	> 6	✗	1.02	6	✓	1.21	> 4	✗	7.74
nagumo[34]	2	$\mathcal{X}$	2	✓	0.13	2	✓	0.14	2	✓	3.50
		$\mathcal{U}, \mathcal{X}$	> 6	✗	1.02	3	✓	0.17	> 4	✗	23.73
lorenz[10]	3	$\mathcal{X}$	6	?	TO	4	✓	72.13	2	?	TO
		$\mathcal{U}, \mathcal{X}$	5	✓	248.88	6	?	TO	2	?	TO
lotka[14]	3	$\mathcal{X}$	> 6	✗	88.97	> 6	✗	27.8	3	?	TO
		$\mathcal{U}, \mathcal{X}$	> 6	✗	0.27	> 6	✗	438.54	3	?	TO

- Expressiveness: Semialgebraic > Polynomial > Incomplete
- Efficiency: Incomplete $\approx$ Polynomial $\gg$ Semialgebraic

- 1 Introduction to Invariant Generation.
- 2 Optimization-based Invariant Generation
- 3 Complete SOS Constraints over Unbounded Domains
- 4 Strong Invariant Generation via Robust Optimization Techniques**
- 5 Challenges

Weak Invariants vs Strong Invariants

- Fix an invariant template parameterized by a .
- The **weak invariant synthesis problem** asks for **one invariant** satisfying the template, i.e., finding a valid parameter assignment.
- The **strong invariant synthesis problem** asks for **a characterization of all possible invariants** satisfying the template, i.e., characterizing the set of valid parameter assignments.

Dealing with BMI Constraints

- We want to solve constraints of the form

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

where we assume domain constraints $\mathbf{x} \in C_{\mathbf{x}}$ and $\mathbf{a} \in C_{\mathbf{a}}$.

- We can treat parameters as variables

$$\exists \mathbf{a}, \forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{a}, \mathbf{x}) \geq 0 \implies l(\mathbf{a}, \mathbf{x}) \geq 0$$

so that the problem becomes a **robust optimization** problem.

- **Question** To characterize the set of solutions for $\mathbf{a}$, denoted by R .

Dealing with BMI Constraints

- We want to solve constraints of the form

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

where we assume domain constraints $\mathbf{x} \in C_{\mathbf{x}}$ and $\mathbf{a} \in C_{\mathbf{a}}$.

- We can treat parameters as variables

$$\exists \mathbf{a}, \forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{a}, \mathbf{x}) \geq 0 \implies l(\mathbf{a}, \mathbf{x}) \geq 0$$

so that the problem becomes a **robust optimization** problem.

- **Question** To characterize the set of solutions for $\mathbf{a}$, denoted by R .

Dealing with BMI Constraints

- We want to solve constraints of the form

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

where we assume domain constraints $\mathbf{x} \in C_{\mathbf{x}}$ and $\mathbf{a} \in C_{\mathbf{a}}$.

- We can treat parameters as variables

$$\exists \mathbf{a}, \forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{a}, \mathbf{x}) \geq 0 \implies l(\mathbf{a}, \mathbf{x}) \geq 0$$

so that the problem becomes a **robust optimization** problem.

- **Question** To characterize the set of solutions for $\mathbf{a}$, denoted by R .

Representing Functions

- R can be expressed as

$$R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}.$$

with

$$\phi(\mathbf{a}) = \begin{cases} -M & \text{if } K(\mathbf{a}) = \emptyset, \\ \max(-M, \sup_{x \in K(\mathbf{a})} l(\mathbf{a}, x)) & \text{otherwise,} \end{cases}$$

where $K(\mathbf{a}) = \{x \in C_x \mid \bigwedge_{i=1}^m k_i(\mathbf{a}, x) \leq 0\}$ and $M > 0$ is a constant.

- The function ϕ is **upper-semicontinuous**, i.e., can be approximated from above by polynomials

Representing Functions

- R can be expressed as

$$R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}.$$

with

$$\phi(\mathbf{a}) = \begin{cases} -M & \text{if } K(\mathbf{a}) = \emptyset, \\ \max(-M, \sup_{x \in K(\mathbf{a})} l(\mathbf{a}, x)) & \text{otherwise,} \end{cases}$$

where $K(\mathbf{a}) = \{x \in C_x \mid \bigwedge_{i=1}^m k_i(\mathbf{a}, x) \leq 0\}$ and $M > 0$ is a constant.

- The function ϕ is **upper-semicontinuous**, i.e., can be approximated from above by polynomials

Under-approximating R

- Under-approximations of set $R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}$ can be obtained by approximating ϕ from above:

$$p_i(\mathbf{a}) \geq \phi(\mathbf{a}) \implies R_i = \{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\} \subseteq R$$

Intuitively, the better p_i approximates ϕ , the less conservative R_i approximates R .

Theorem (Lasserre, 2015)

Assume that $\partial R = \{\mathbf{a} \mid \phi(\mathbf{a}) = 0\}$ has Lebesgue measure zero. If $p_i \geq \phi$ converges to ϕ w.r.t. L1-norm, then $R_i \subseteq R$ and:

$$\lim_{i \rightarrow \infty} \mu(R \setminus R_i) = 0 \quad (\mu \text{ Lebesgue measure})$$

Under-approximating R

- Under-approximations of set $R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}$ can be obtained by approximating ϕ from above:

$$p_i(\mathbf{a}) \geq \phi(\mathbf{a}) \implies R_i = \{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\} \subseteq R$$

Intuitively, the better p_i approximates ϕ , the less conservative R_i approximates R .

Theorem (Lasserre, 2015)

Assume that $\partial R = \{\mathbf{a} \mid \phi(\mathbf{a}) = 0\}$ has Lebesgue measure zero. If $p_i \geq \phi$ converges to ϕ w.r.t. $L1$ -norm, then $R_i \subseteq R$ and:

$$\lim_{i \rightarrow \infty} \mu(R \setminus R_i) = 0 \quad (\mu \text{ Lebesgue measure})$$

Under-approximating R

- Under-approximations of set $R = \{\mathbf{a} \mid \phi(\mathbf{a}) \leq 0\}$ can be obtained by approximating ϕ from above:

$$p_i(\mathbf{a}) \geq \phi(\mathbf{a}) \implies R_i = \{\mathbf{a} \mid p_i(\mathbf{a}) \leq 0\} \subseteq R$$

Intuitively, the better p_i approximates ϕ , the less conservative R_i approximates R .

Theorem (Lasserre, 2015)

Assume that $\partial R = \{\mathbf{a} \mid \phi(\mathbf{a}) = 0\}$ has Lebesgue measure zero. If $p_i \geq \phi$ converges to ϕ w.r.t. $L1$ -norm, then $R_i \subseteq R$ and:

$$\lim_{i \rightarrow \infty} \mu(R \setminus R_i) = 0 \quad (\mu \text{ Lebesgue measure})$$

Under-approximation by SDP

- **Constraints:** For $p_i(\mathbf{a})$ to approximate $\phi(\mathbf{a})$ from above, we need constraints $p_i(\mathbf{a}) \geq -M$ and $p_i(\mathbf{a}) \geq l(\mathbf{a}, \mathbf{x})$ for all $(\mathbf{a}, \mathbf{x})$.
- **Objective Function:** To obtain a more accurate approximation of $\phi(\mathbf{a})$, we search for p_i with minimal L1-norm over C_a :

$$\int_{C_a} p_i(\mathbf{a}) d\mu(\mathbf{a}) = \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \text{ with } \gamma_{\alpha} = \int_{C_a} a^{\alpha} d\mu(\mathbf{a})$$

- SDP formulation

$$\begin{aligned} \min \quad & \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \\ \text{s.t.} \quad & p_i(\mathbf{a}) - l(\mathbf{a}, \mathbf{x}) \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times K_i(\mathbf{a}) \\ & p_i(\mathbf{a}) + M \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times C_x. \end{aligned}$$

Under-approximation by SDP

- **Constraints:** For $p_i(\mathbf{a})$ to approximate $\phi(\mathbf{a})$ from above, we need constraints $p_i(\mathbf{a}) \geq -M$ and $p_i(\mathbf{a}) \geq l(\mathbf{a}, \mathbf{x})$ for all $(\mathbf{a}, \mathbf{x})$.
- **Objective Function:** To obtain a more accurate approximation of $\phi(\mathbf{a})$, we search for p_i with minimal L1-norm over C_a :

$$\int_{C_a} p_i(a) d\mu(a) = \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \text{ with } \gamma_{\alpha} = \int_{C_a} a^{\alpha} d\mu(a)$$

- SDP formulation

$$\begin{aligned} \min \quad & \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \\ \text{s.t.} \quad & p_i(\mathbf{a}) - l(\mathbf{a}, \mathbf{x}) \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times K_i(\mathbf{a}) \\ & p_i(\mathbf{a}) + M \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times C_x. \end{aligned}$$

Under-approximation by SDP

- **Constraints:** For $p_i(\mathbf{a})$ to approximate $\phi(\mathbf{a})$ from above, we need constraints $p_i(\mathbf{a}) \geq -M$ and $p_i(\mathbf{a}) \geq l(\mathbf{a}, \mathbf{x})$ for all $(\mathbf{a}, \mathbf{x})$.
- **Objective Function:** To obtain a more accurate approximation of $\phi(\mathbf{a})$, we search for p_i with minimal L1-norm over C_a :

$$\int_{C_a} p_i(a) d\mu(a) = \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \text{ with } \gamma_{\alpha} = \int_{C_a} a^{\alpha} d\mu(a)$$

- SDP formulation

$$\begin{aligned} \min \quad & \sum_{\alpha} \gamma_{\alpha} p_{\alpha}^i \\ \text{s.t.} \quad & p_i(\mathbf{a}) - l(\mathbf{a}, \mathbf{x}) \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times K_i(\mathbf{a}) \\ & p_i(\mathbf{a}) + M \geq 0, \quad \forall (\mathbf{a}, \mathbf{x}) \in C_a \times C_{\mathbf{x}}. \end{aligned}$$

Weak Completeness

- The under-approximations $R_i = \{\mathbf{a} \in C_a \mid p_i(\mathbf{a}) \leq 0\}$ converge to R w.r.t. L1 measure.

Theorem (Weak Completeness)

If $\partial R = \{a \mid \phi(a) = 0\}$ has measure zero and the valid set R has positive measure, our algorithm can find a non-empty under-approximation.

Masked Templates

- **Question** What if the valid set R is zero-measured?
- This can happen when the template includes **several** polynomial inequalities and polynomial equalities.

Code 4 freire1

```
// Program variables:  $(x, y, r) \in \mathbb{R}^n$ 
// Precondition:  $Pre = \{(x, y, r) \mid -y \leq 0, x = y/2, r = 0\}$ 
// Invariant template:  $\{y = \text{poly}[(x, r), 2], -x \leq 0\}$ 
while  $(r - x \leq 0)$  { // Real invariant:  $y = 2x + r^2 - r \wedge x \geq 0$ 
     $x \leftarrow x - r;$ 
     $y \leftarrow y;$ 
     $r \leftarrow r + 1;$ 
}
// Postcondition:  $Post = \{y - r^2 - r \leq 0, r^2 - r - y \leq 0\}$ 
```

Masked Templates

- In many practical programs, variables can be divided into two groups
 - ▶ **Core variables:** x' depends only on x . ($\longrightarrow$ local variables)
 - ▶ **Non-core variables:** y' depends on both x and y . ($\longrightarrow$ input variables)

- We define **masked templates:** to be of the form

$$\{y = I_r(x; a), I_t(x, y) \leq 0\}$$

- ▶ equalities: non-core variable = polynomial of core variables (contain parameters)
 - ▶ inequalities: no parameters
- By using **Variable Substitution**, the BMI can be transformed into an equivalent LMI.

Masked Templates

- In many practical programs, variables can be divided into two groups
 - ▶ **Core variables:** x' depends only on x . ($\longrightarrow$ local variables)
 - ▶ **Non-core variables:** y' depends on both x and y . ($\longrightarrow$ input variables)

- We define **masked templates**: to be of the form

$$\{y = I_r(x; a), I_t(x, y) \leq 0\}$$

- ▶ equalities: non-core variable = polynomial of core variables (contain parameters)
 - ▶ inequalities: no parameters
- By using **Variable Substitution**, the BMI can be transformed into an equivalent LMI.

Masked Templates

- In many practical programs, variables can be divided into two groups
 - ▶ **Core variables:** x' depends only on x . ($\longrightarrow$ local variables)
 - ▶ **Non-core variables:** y' depends on both x and y . ($\longrightarrow$ input variables)

- We define **masked templates**: to be of the form

$$\{y = I_r(x; a), I_t(x, y) \leq 0\}$$

- ▶ equalities: non-core variable = polynomial of core variables (contain parameters)
 - ▶ inequalities: no parameters
- By using **Variable Substitution**, the BMI can be transformed into an equivalent LMI.

Experiments

Table 1. Experimental results over Cluster benchmarks.

Benchmark	n_x	n_a	Cluster (Alg. 1)			POLYSYNTH [30]		Z3 [23]	
			D	result	time	result	time	result	time
freire1-1	3	2	1	✓	0.7s	✓	3.6s	✓	0.1s
freire1-2	3	2	2	✓	4.2s	TO	>600s	TO	>600s
freire1-3	3	2	1	✓	0.8s	✓	3.1s	✓	61.0s
cohencu-1	2	3	2	✓	5.4s	TO	>600s	✓	0.2s
cohencu-2	2	3	3	✓	9.4s	TO	>600s	TO	>600s
cohencu-3	2	3	3	✓	11.2s	TO	>600s	✓	0.2s
Exmp. 1	2	2	3	✓	3.0s	TO	>600s	TO	>600s
circuit [4]	2	2	2	✓	2.8s	TO	>600s	TO	>600s
unicycle [80]	2	3	≥ 7	TO	>600s	TO	>600s	TO	>600s
overview [21]	2	3	≥ 8	TO	>600s	TO	>600s	TO	>600s

n_x, n_a : the number of variables and parameters, respectively. For the Cluster algorithm: D is the smallest degree upper bound such that a non-empty under-approximation can be found, “time” includes the solving SDP time and the posterior verification time. TO: timeout, 600s. The boldface marks the winner.

Experiments

Table 2. Experimental results over masked template benchmarks.

Benchmark	n_y	n_z	n_a	k	Mask (Alg. 2)		POLYSYNTH [30]		LIPuS [92]	
					result	time	result	time	result	time†
berkeley	4	1	5	4	✓	3.8s	TO	>600s	TO	>600s
cohencu	1	3	12	1	✓	1.7s	TO	>600s	TO	>600s
cohendiv	4	1	15	1	✓	0.6s	✓	6.8s	(✓)	(142.0s)
euclidex2	6	2	56	2	✓	4.0s	TO	>600s	TO	>600s
fermat2	4	1	15	2	✓	0.9s	✓	10.3s	TO	>600s
firefly	4	1	5	7	✓	10.9s	TO	>600s	TO	>600s
freire1	2	1	6	1	✓	0.7s	✓	70.4s	(✓)	(460.0s)
freire2	4	-	-	1	NS	-	TO	>600s	NS	-
illinois	4	1	5	10	✓	17.0s	TO	>600s	TO	>600s
lcm	6	1	28	2	✓	1.3s	✓	17.1s	TO	>600s
mannadiv	4	1	15	3	✓	1.2s	TO	>600s	TO	>600s
mesi	4	1	5	4	✓	4.5s	TO	>600s	(✓)	(592.5s)
moesi	5	1	6	5	✓	7.7s	TO	>600s	(✓)	(117.2s)
petter	1	1	7	1	✓	0.6s	✓	4.4s	TO	>600s
readerswriters	5	1	21	4	✓	3.4s	TO	>600s	TO	>600s
sqrt	4	-	-	1	NS	-	TO	>600s	(✓)	(421.0s)
wensley	7	-	-	2	NS	-	TO	>600s	NS	-
z3sqrt	4	1	15	2	✓	1.1s	TO	>600s	NS	-
sum2power10	2	1	66	1	✓	8.6s	TO	>600s	TO	>600s
sum2power15	2	1	136	1	✗	200.3s	TO	>600s	TO	>600s
sum3power6	3	1	84	1	✓	9.9s	TO	>600s	TO	>600s
sum3power8	3	1	102	1	✓	102.4	TO	>600s	TO	>600s
sum5power4	5	1	126	1	✓	26.6s	TO	>600s	TO	>600s
sum5power5	5	1	252	1	TO	>600s	TO	>600s	TO	>600s
sum8power3	8	1	165	1	✓	184.6s	TO	>600s	TO	>600s

n_y, n_z, n_a : the number of core variables, non-core variables, and parameters in templates, respectively. k : the number of branches in loop body. For the Mask algorithm, "time" includes the solving SDP time and the posterior verification time. TO: timeout, 600s. NS: unsupported benchmarks where there are no non-core variables (Mask algorithm) or containing floating-point variables (not allowed in LIPuS). ✗: fail to synthesize an invariant or unsound invariant. †: the results in the last column were provided by the authors of LIPuS using their

- 1 Introduction to Invariant Generation.
- 2 Optimization-based Invariant Generation
- 3 Complete SOS Constraints over Unbounded Domains
- 4 Strong Invariant Generation via Robust Optimization Techniques
- 5 Challenges**

Template Setting

- We have assumed that all templates are of the form $I(\mathbf{x}; \mathbf{a}) \leq 0$ with $I \in \mathbb{R}[\mathbf{x}]$.
- **Question 1 (polynomial-level)** What about piece-wise polynomial $I_1(\mathbf{x}; \mathbf{a}_1) \leq 0 \wedge I_2(\mathbf{x}; \mathbf{a}_2) \leq 0$? How to decide when we need equality constraints?

Code 4 freire1

```
// Program variables:  $(x, y, r) \in \mathbb{R}^n$ 
// Precondition:  $Pre = \{(x, y, r) \mid -y \leq 0, x = y/2, r = 0\}$ 
// Invariant template:  $\{y = \text{poly}[(x, r), 2], -x \leq 0\}$ 
while  $(r - x \leq 0)$  { // Real invariant:  $y = 2x + r^2 - r \wedge x \geq 0$ 
     $x \leftarrow x - r;$ 
     $y \leftarrow y;$ 
     $r \leftarrow r + 1;$ 
}
// Postcondition:  $Post = \{y - r^2 - r \leq 0, r^2 - r - y \leq 0\}$ 
```

Template Setting

- When we set a template $I(\mathbf{x}; \mathbf{a}) \leq 0$, we include all monomials up to a certain degree.
- **Question 2 (monomial-level)** How to set a **sparse** polynomial template?
- Given a basic semialgebraic set K whose defining polynomials have certain sparsity pattern, to find a polynomial p non-negative over K with the number of monomials as small as possible.

Non-Convex Optimization

- **Question 3:** How to solve the BMI constraints translated from invariant generation?

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

or

find $\mathbf{a}, \mathbf{b}$

$$- B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(\mathbf{x}; \mathbf{b}) B(\mathbf{x}; \mathbf{a}) - \mathfrak{L}_f B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(\mathbf{x}; \mathbf{a}) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- **Question 4:** How to learn from a failure, i.e. why LMI or BMI is not solvable?

Thanks! & Questions?

Non-Convex Optimization

- **Question 3:** How to solve the BMI constraints translated from invariant generation?

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

or

find $\mathbf{a}, \mathbf{b}$

$$- B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(\mathbf{x}; \mathbf{b}) B(\mathbf{x}; \mathbf{a}) - \mathcal{L}_f B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(\mathbf{x}; \mathbf{a}) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- **Question 4:** How to learn from a failure, i.e. why LMI or BMI is not solvable?

Thanks! & Questions?

Non-Convex Optimization

- **Question 3:** How to solve the BMI constraints translated from invariant generation?

$$\forall \mathbf{x}. \bigwedge_{i=1}^m k_i(\mathbf{x}; \mathbf{a}) \geq 0 \implies l(\mathbf{x}; \mathbf{a}) \geq 0$$

or

find $\mathbf{a}, \mathbf{b}$

$$- B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_i),$$

$$\lambda(\mathbf{x}; \mathbf{b}) B(\mathbf{x}; \mathbf{a}) - \mathcal{L}_f B(\mathbf{x}; \mathbf{a}) \in \mathbf{QM}(\mathcal{X}_d)$$

$$B(\mathbf{x}; \mathbf{a}) - \epsilon_e \in \mathbf{QM}(\mathcal{X}_u)$$

- **Question 4:** How to learn from a failure, i.e. why LMI or BMI is not solvable?

Thanks! & Questions?



Huang, Lei, Jiawang Nie, and Ya-Xiang Yuan (2023).

“Homogenization for polynomial optimization with unbounded sets”.
In: *Mathematical Programming* 200.1, pp. 105–145.



Lasserre, Jean B (2015). “Tractable approximations of sets defined with quantifiers”. In: *Mathematical Programming* 151.2, pp. 507–527.



Nie, Jiawang (2012). “Discriminants and nonnegative polynomials”. In: *Journal of Symbolic Computation* 47.2, pp. 167–191.



Wu, Hao, Shenghua Feng, Ting Gan, Jie Wang, Bican Xia, and Naijun Zhan (2024). “On Completeness of SDP-Based Barrier Certificate Synthesis over Unbounded Domains”. In: *FM (2)*. Vol. 14934. LNCS. Springer, pp. 248–266.



Wu, Hao, Qiuye Wang, Bai Xue, Naijun Zhan, Lihong Zhi, and Zhi-Hong Yang (Feb. 2025). “Synthesizing Invariants for Polynomial Programs by Semidefinite Programming”. In: *ACM Trans. Program. Lang. Syst.* 47.1. ISSN: 0164-0925. DOI: 10.1145/3708559.